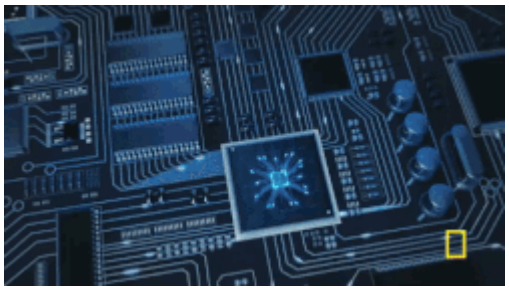
Schaltnetze

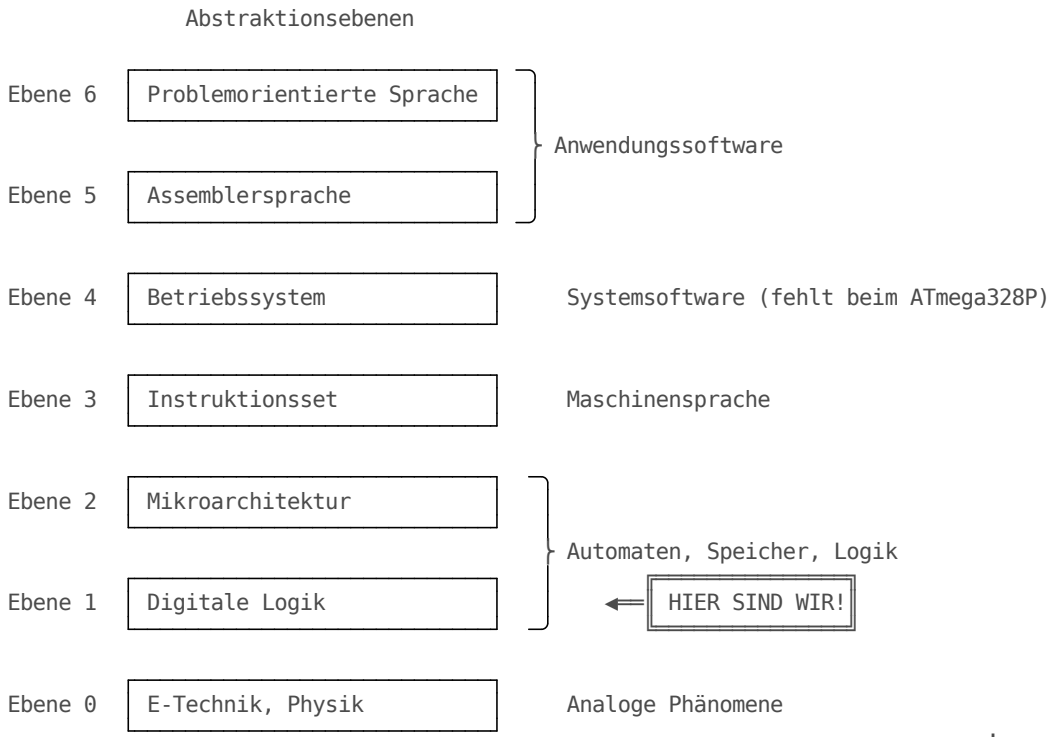
Parameter	Kursinformationen
Veranstaltung:	`Digitale Systeme / Eingebettete Systeme`
Semester:	`Wintersemester 2026/27`
Hochschule:	`Technische Universität Freiberg`
Inhalte:	`Verfahren von Quine-McCluskey, Realisierung von Schaltnetzen`
Link auf GitHub:	https://github.com/TUBAF-IfI-LiaScript/VL_EingebetteteSysteme/blob/master/04_Schaltnetze.md
Autoren:	Sebastian Zug & André Dietrich & Fabian Bär & Copilot & Claude Code (Opus 5.5)



Lernziele: Nach dieser Vorlesung können Sie

- das Verfahren nach Quine-McCluskey anwenden: Primimplikanten bestimmen und mit der Primimplikantentafel eine minimale Überdeckung auswählen,
- eine minimierte Schaltfunktion als zweistufiges Schaltnetz umsetzen – mit UND, ODER und NICHT oder nur mit NAND-Gattern,
- erklären, wann PROM und PAL zur Umsetzung von Schaltnetzen eingesetzt werden,
- Glitches als Folge unterschiedlicher Signallaufzeiten erkennen und mit einem zusätzlichen Term aus dem Karnaugh-Veitch-Diagramm vermeiden.

🗺️ Diese Vorlesung in der Kurslandkarte →



Rückblick: Hausaufgabe aus Vorlesung 03

Wir beginnen mit Teil B der Hausaufgabe aus der letzten Vorlesung. Beide Aufgaben drehen sich um Segment d der Siebensegmentanzeige und um die Frage, was Don't-Cares in der Praxis bedeuten.

Aufgabe 4 aus Vorlesung 03: Ändern Sie `Minimize.py` so, dass es Segment `d` minimiert: Tragen Sie die Einsen in `minterms` ein und übergeben Sie die Eingaben 10 bis 15 als dritten Parameter von `SOPform`. Vergleichen Sie das Ergebnis mit dem Abschnitt zu den Don't-Cares. Weicht es ab, warum darf es das? Lassen Sie danach den dritten Parameter weg. Was ändert sich, und für welche Eingaben liefert die Funktion nun andere Werte?

Segment_d.py

```python
from sympy.logic import SOPform
from sympy import symbols
w, x, y, z = symbols('w x y z')

# jede Zeile: eine Zeile der Wahrheitstafel (w, x, y, z)
minterms  = [[0, 0, 0, 0], [0, 0, 1, 0], [0, 0, 1, 1], [0, 1, 0, 1],
             [0, 1, 1, 0], [1, 0, 0, 0], [1, 0, 0, 1]]
dontcares = [[1, 0, 1, 0], [1, 0, 1, 1], [1, 1, 0, 0],
             [1, 1, 0, 1], [1, 1, 1, 0], [1, 1, 1, 1]]

print("mit Don't-Cares: ", SOPform([w, x, y, z], minterms, dontcares))
print("ohne Don't-Cares:", SOPform([w, x, y, z], minterms))
```

```
mit Don't-Cares:  w | (y & ~x) | (y & ~z) | (~x & ~z) | (x & z & ~y)
ohne Don't-Cares: (w & ~x & ~y) | (y & ~w & ~x) | (y & ~w & ~z) | (~w &
~x & ~z) | (x & z & ~w & ~y)
```

- **Mit Don't-Cares** liefert SymPy dieselben fünf Terme wie das KV-Diagramm, $d = w + y\bar{z} + \bar{x}y + x\bar{y}z + \bar{x}\bar{z}$, also 10 Literale. Diese Minimalform ist eindeutig, eine Abweichung kann nur in der Reihenfolge der Terme liegen.
- **Ohne Don't-Cares** sind es ebenfalls fünf Terme, aber 16 Literale: $d = w\bar{x}\bar{y} + \bar{w}\bar{x}y + \bar{w}y\bar{z} + \bar{w}\bar{x}\bar{z} + \bar{w}x\bar{y}z$. Hier gibt es eine zweite, gleich gute Form mit $\bar{x}\bar{y}\bar{z}$ statt $\bar{w}\bar{x}\bar{z}$. SymPy darf jede der beiden ausgeben.
- **Für die Ziffern 0 bis 9** sind beide Formen gleich. Für die Eingaben 10 bis 15 liefert die Form mit Don't-Cares immer 1, weil schon der Term w dort 1 ist. Die Form ohne Don't-Cares liefert dort immer 0.

> **Aufgabe 5 aus Vorlesung 03:** Bauen Sie Segment `d` selbst auf, im Simulator BEAST, der ohne Installation im Browser läuft. Ziehen Sie aus „Basic Components“ vier `Toggle` für w, x, y, z, ein `4 bit 7 Segment Display` für die Ziffer, die Gatter `NOT`, `AND` und `OR` sowie eine `LED` für `d` auf die Arbeitsfläche und verdrahten Sie sie nach der Gleichung aus dem Abschnitt zu den Don't-Cares. Prüfen Sie die Ziffern 0 bis 9: Leuchtet die LED genau dann, wenn das untere Segment der Ziffer leuchtet? Was zeigen LED und Anzeige bei den Eingaben 10 bis 15?

Für die Ziffern 0 bis 9 leuchtet die LED genau dann, wenn das untere Segment leuchtet, die Schaltung stimmt also. Für die Eingaben 10 bis 15 leuchtet die LED immer, denn der Term w allein liefert dort schon 1. Die Anzeige zeigt dort keine Dezimalziffer mehr. Genau das meint der Satz aus der letzten Vorlesung: Ein Don't-Care ist eine Annahme. Kommen diese Eingaben doch vor, gibt die Schaltung einen Wert aus, den niemand geplant hat.

Das Video fasst das Vorgehen im KV-Diagramm mit vier Variablen noch einmal zusammen:



Beispielanwendung

Heute begleitet uns eine Kaffeemaschine. Sie meldet über vier Leitungen ihren Zustand, und eine Fehler-LED soll in bestimmten Zuständen leuchten. Bevor wir rechnen, minimieren Sie die Funktion selbst im KV-Diagramm.

Nehmen wir an, dass wir die Logik einer Kaffeemaschine umsetzen wollen. Ihre Steuerung wertet Sensoren an Heizplatte, Füllstand und Druck aus und meldet den aktuellen Zustand als vierstellige Binärzahl $x_3 x_2 x_1 x_0$, also als Nummer von 0 bis 15.

Die Variable y gibt einen Fehlerzustand wieder und ist zum Beispiel mit einer LED verknüpft. In den Zuständen 6 bis 14 soll diese leuchten.



x_3	x_2	x_1	x_0	y	**Zustand**
0	0	0	0	0	0 - Initialisierung
0	0	0	1	0	1 - Heizplatte erwärmen
0	0	1	0	0	2 - Wasserkocher an
0	0	1	1	0	3 -
0	1	0	0	0	4 -
0	1	0	1	0	5 -
0	1	1	0	1	6 - Wasser fehlt
0	1	1	1	1	7 - Kaffeefach geöffnet
1	0	0	0	1	8 - Druckabfall
1	0	0	1	1	9 - …
1	0	1	0	1	10 -
1	0	1	1	1	11 -
1	1	0	0	1	12 -

1	1	0	1	1	13 -
1	1	1	0	1	14 - Wartung fällig
1	1	1	1	0	15 - Kaffee fertig

Zum Aufwärmen: Minimieren Sie y selbst im KV-Diagramm. Die Einsen sind schon eingetragen, die kleine Zahl im Feld ist die Zeile der Wahrheitstafel. Anders als in den Tabellen weiter unten stehen $x_3 x_2$ hier in den Spalten.

Felder eintragen | Schleifen bilden | DNF (Einsen) | KNF (Nullen)

	$\bar{x}_3\bar{x}_2$	$\bar{x}_3 x_2$	$x_3 x_2$	$x_3\bar{x}_2$
$\bar{x}_1\bar{x}_0$	0 0	0 4	1 12	1 8
$\bar{x}_1 x_0$	0 1	0 5	1 13	1 9
$x_1 x_0$	0 3	1 7	0 15	1 11
$x_1\bar{x}_0$	0 2	1 6	1 14	1 10

Schleife übernehmen | Lösung zeigen | Zurücksetzen

Felder anklicken, die eine Schleife bilden.

f = ?

Noch nicht überdeckt: 9 Einsen.

Vier Schleifen genügen: drei Vierer-Schleifen mit x3 gleich 1 und eine Zweier-Schleife für die Zustände 6 und 7. Im Diagramm ging das schnell. Heute geht es um ein Verfahren, das ohne Bild auskommt und deshalb auch für zehn oder zwanzig Variablen funktioniert.

Lösung: $y = \bar{x}_3 x_2 x_1 + x_3\bar{x}_2 + x_3\bar{x}_1 + x_3\bar{x}_0$, also 4 Terme mit 9 Literalen. Die drei Vierer-Schleifen liegen alle in den Spalten mit $x_3 = 1$, die Zweier-Schleife deckt die Zustände 6 und 7 ab.

Im Diagramm ging das schnell. Mit fünf Variablen wird es unübersichtlich, ab sechs geht es kaum noch. Heute lernen Sie ein Verfahren, das ohne Bild auskommt und sich programmieren lässt: Quine-McCluskey.

Wie muss also die Schaltung für diese Aufgabe umgesetzt werden? Wir gehen in drei Schritten vor: Gleichungen aufstellen, vereinfachen, umsetzen.

Schritt 1: Aufstellen der Gleichungen

Stellen Sie die Terme zuerst selbst auf, wie in der letzten Vorlesung: Jede Zeile mit einer Eins liefert einen Minterm, jede Zeile mit einer Null einen Maxterm.

Aus Vorlesung 03 kennen Sie den Weg: Jede Zeile mit $y = 1$ liefert einen Minterm der KDNF, jede Zeile mit $y = 0$ einen Maxterm der KKNF.

x_3	x_2	x_1	x_0	y	**Minterm**	**Maxterm**
0	0	0	0	0	???	???
0	0	0	1	0		
0	0	1	0	0		
0	0	1	1	0		
0	1	0	0	0		
0	1	0	1	0		
0	1	1	0	1		
0	1	1	1	1		
1	0	0	0	1		
1	0	0	1	1		
1	0	1	0	1		
1	0	1	1	1		
1	1	0	0	1		
1	1	0	1	1		
1	1	1	0	1		
1	1	1	1	0		

Hier die Lösung. Neun Zeilen mit einer Eins ergeben neun Minterme, sieben Zeilen mit einer Null sieben Maxterme. Wir arbeiten mit der KDNF weiter.

x_3	x_2	x_1	x_0	y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Schritt 2: Vereinfachen der Minterme - Analytische Lösung

Zuerst vereinfachen wir von Hand, so wie in Vorlesung 02. Wir suchen Paare von Mintermen, die sich nur in einer Variablen unterscheiden.

$$
\begin{aligned}
y = \quad & \bar{x}_3 x_2 x_1 \bar{x}_0 + \bar{x}_3 x_2 x_1 x_0 + \\
& x_3 \bar{x}_2 \bar{x}_1 \bar{x}_0 + x_3 \bar{x}_2 \bar{x}_1 x_0 + \\
& x_3 \bar{x}_2 x_1 \bar{x}_0 + x_3 \bar{x}_2 x_1 x_0 + \\
& x_3 x_2 \bar{x}_1 \bar{x}_0 + x_3 x_2 \bar{x}_1 x_0 + \\
& x_3 x_2 x_1 \bar{x}_0
\end{aligned}
$$

Wie gehen Sie vor? Wir suchen Paare von Mintermen, die sich lediglich in einer Variablen unterscheiden, und fassen diese mit dem Distributivgesetz und $x + \bar{x} = 1$ zusammen.

$$
\begin{aligned}
\bar{x}_3 x_2 x_1 \bar{x}_0 + \bar{x}_3 x_2 x_1 x_0 \quad & = \bar{x}_3 x_2 x_1 (\bar{x}_0 + x_0) \\
& = \bar{x}_3 x_2 x_1 (1) \\
& = \bar{x}_3 x_2 x_1
\end{aligned}
$$

Ein Minterm passt zu zwei Partnern. Wir nehmen ihn deshalb zweimal auf, das erlaubt das Idempotenzgesetz.

Erste Stufe der Vereinfachung

Zunächst nehmen wir einen der Minterme zweimal auf (Idempotenzgesetz: $a + a = a$), so dass sich jede Zeile zusammenfassen lässt.

$$
\begin{aligned}
y = \quad & \bar{x}_3 x_2 x_1 \bar{x}_0 + \bar{x}_3 x_2 x_1 x_0 + \\
& x_3 \bar{x}_2 \bar{x}_1 \bar{x}_0 + x_3 \bar{x}_2 \bar{x}_1 x_0 + \\
& x_3 \bar{x}_2 x_1 \bar{x}_0 + x_3 \bar{x}_2 x_1 x_0 + \\
& x_3 x_2 \bar{x}_1 \bar{x}_0 + x_3 x_2 \bar{x}_1 x_0 + \\
& x_3 \bar{x}_2 x_1 \bar{x}_0 + x_3 x_2 x_1 \bar{x}_0
\end{aligned}
$$

Damit ergibt sich die Gleichung nach der ersten Vereinfachungsstufe zu

Aus neun Mintermen sind fünf Terme mit je drei Variablen geworden.

$$
y = \quad \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 \bar{x}_1 + x_3 \bar{x}_2 x_1 + x_3 x_2 \bar{x}_1 + x_3 x_1 \bar{x}_0
$$

In der zweiten Stufe wiederholen wir dasselbe mit den neuen Termen.

Zweite Stufe der Vereinfachung

$$
\begin{aligned}
y = \quad & \bar{x}_3 x_2 x_1 + \\
& x_3 \bar{x}_2 \bar{x}_1 + \\
& x_3 \bar{x}_2 x_1 + \\
& x_3 x_2 \bar{x}_1 + \\
& x_3 x_1 \bar{x}_0
\end{aligned}
$$

Übrig bleiben vier Terme. Vergleichen Sie mit Ihrem KV-Diagramm vom Anfang: Der letzte Term hat eine Variable zu viel.

$$
y = \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 + x_3 \bar{x}_1 + x_3 x_1 \bar{x}_0
$$

Dabei wird $x_3 \bar{x}_2 \bar{x}_1$ zweimal verwendet, einmal mit $x_3 \bar{x}_2 x_1$ zu $x_3 \bar{x}_2$ und einmal mit $x_3 x_2 \bar{x}_1$ zu $x_3 \bar{x}_1$. Das erlaubt wieder das Idempotenzgesetz.

Die Gegenprobe mit SymPy bestätigt den Verdacht. Die Umformung von Hand hat eine Möglichkeit übersehen.

Gegenprobe I

Minimize.py

```python
from sympy.logic import SOPform
from sympy import symbols
x3, x2, x1, x0 = symbols('x3 x2 x1 x0')

minterms = [[0, 1, 1, 0],
            [0, 1, 1, 1],
            [1, 0, 0, 0],
            [1, 0, 0, 1],
            [1, 0, 1, 0],
            [1, 0, 1, 1],
            [1, 1, 0, 0],
            [1, 1, 0, 1],
            [1, 1, 1, 0]]
result = SOPform([x3, x2, x1, x0], minterms)
print(result)
```

```
(x3 & ~x0) | (x3 & ~x1) | (x3 & ~x2) | (x1 & x2 & ~x3)
```

SymPy findet eine kompaktere Form, nämlich genau die aus Ihrem KV-Diagramm vom Anfang.

Schauen wir uns die Funktion im Karnaugh-Diagramm an! Es ist dieselbe Funktion wie beim Aufwärmen. Mit „Lösung zeigen“ sehen Sie die Schleifen, die SymPy gefunden hat.

Felder eintragen | Schleifen bilden | DNF (Einsen) | KNF (Nullen)

	$\bar{x}_3\bar{x}_2$	$\bar{x}_3 x_2$	$x_3 x_2$	$x_3\bar{x}_2$
$\bar{x}_1\bar{x}_0$	0 (0)	0 (4)	1 (12)	1 (8)
$\bar{x}_1 x_0$	0 (1)	0 (5)	1 (13)	1 (9)
$x_1 x_0$	0 (3)	1 (7)	0 (15)	1 (11)
$x_1\bar{x}_0$	0 (2)	1 (6)	1 (14)	1 (10)

Schleife übernehmen | Lösung zeigen | Zurücksetzen

Felder anklicken, die eine Schleife bilden.

$f = ?$

Noch nicht überdeckt: 9 Einsen.

$$
\begin{aligned}
y_{Karnaugh} &= \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 + x_3 \bar{x}_1 + x_3 \bar{x}_0 \\
y_{Analytic} &= \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 + x_3 \bar{x}_1 + x_3 x_1 \bar{x}_0
\end{aligned}
$$

Warum „übersieht“ unsere analytische Lösung die mögliche Minimierung im letzten Term?

> Weil wir uns in der Vereinfachung der ersten Stufe mit einem Teilergebnis zufriedengegeben haben, das uns in der zweiten Stufe für einen weiteren Zusammenfassungsschritt fehlt.

Welche Kombinationen sind zusätzlich möglich?

$$
\begin{aligned}
y = \quad & \bar{x}_3 x_2 x_1 \bar{x}_0 + \bar{x}_3 x_2 x_1 x_0 + \\
& x_3 \bar{x}_2 \bar{x}_1 \bar{x}_0 + x_3 \bar{x}_2 \bar{x}_1 x_0 + \\
& x_3 \bar{x}_2 x_1 \bar{x}_0 + x_3 \bar{x}_2 x_1 x_0 + \\
& x_3 x_2 \bar{x}_1 \bar{x}_0 + x_3 x_2 \bar{x}_1 x_0 + \\
& x_3 \bar{x}_2 x_1 \bar{x}_0 + x_3 x_2 x_1 \bar{x}_0
\end{aligned}
$$

$$
y = \quad \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 \bar{x}_1 + x_3 \bar{x}_2 x_1 + x_3 x_2 \bar{x}_1 + x_3 x_1 \bar{x}_0 + x_3 \bar{x}_1 \bar{x}_0
$$

Daraus ergibt sich dann auf der zweiten Stufe die „zusätzliche“ Minimierungsoption, die die beiden letzten Terme zusammenfasst.

$$
\begin{aligned}
y = \quad & \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 \bar{x}_1 + x_3 \bar{x}_2 x_1 + x_3 x_2 \bar{x}_1 + \underbrace{x_3 x_1 \bar{x}_0 + x_3 \bar{x}_1 \bar{x}_0}_{x_3 \bar{x}_0} \\
y = \quad & \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 + x_3 \bar{x}_1 + x_3 \bar{x}_0
\end{aligned}
$$

> Erkenntnisse:
>
> - Das Karnaugh-Veitch-Diagramm zeigt mögliche Minimierungspotentiale auf, hinsichtlich der Bildung der Schleifen können unterschiedliche Strategien zum Tragen kommen.
> - Offenkundig brauchen wir ein systematischeres Vorgehen bei der Vereinfachung, das alle Kombinationen möglicher Terme berücksichtigt.

Verfahren nach Quine-McCluskey

Quine-McCluskey macht dasselbe wie die Umformung von Hand, aber vollständig: Es probiert jedes Paar, das sich zusammenfassen lässt, und übersieht deshalb nichts. Damit eignet es sich für den Rechner.

Das Verfahren bezieht sich zunächst nur auf Funktionsdarstellungen in kanonischer disjunktiver Normalform (KDNF) und zielt darauf ab eine systematische Minimierung der Minterme durchzuführen.

Ermittlung der Primimplikanten

Wir sortieren die Minterme nach der Zahl ihrer Einsen. Nur Terme aus benachbarten Gruppen können sich in genau einer Stelle unterscheiden, das spart viele Vergleiche.

Die Minterme werden in tabellarischer Form entsprechend der Zahl der „1“en sortiert. Es gibt 1 Eintrag mit einer 1 (m_8) und jeweils 4 mit 2 oder 3 Einsen.

	x_3	x_2	x_1	x_0	y
	0	0	0	0	0
	0	0	0	1	0
	0	0	1	0	0
	0	0	1	1	0
	0	1	0	0	0
	0	1	0	1	0
m_6	0	1	1	0	1
m_7	0	1	1	1	1
m_8	1	0	0	0	1
m_9	1	0	0	1	1
m_{10}	1	0	1	0	1
m_{11}	1	0	1	1	1
m_{12}	1	1	0	0	1
m_{13}	1	1	0	1	1
m_{14}	1	1	1	0	1
	1	1	1	1	0

m_x	x_3	x_2	x_1	x_0	OK
m_8	1	0	0	0	.
m_6	0	1	1	0	.
m_9	1	0	0	1	.
m_{10}	1	0	1	0	.
m_{12}	1	1	0	0	.
m_7	0	1	1	1	.
m_{11}	1	0	1	1	.
m_{13}	1	1	0	1	.
m_{14}	1	1	1	0	.

Schreibweise: m_8, m_9 bezeichnet den Term, der aus m_8 und m_9 entsteht. In der Tabelle steht er als `100-`, der Strich markiert die Variable, die wegfällt, hier also $x_3\bar{x}_2\bar{x}_1$. In der Spalte „OK“ bedeutet $*$: Dieser Term ist in einen größeren Term eingegangen. Was am Ende ohne $*$ übrig bleibt, nennen wir

Primimplikant und kürzen es mit P ab.

In einem zweiten Schritt werden die sortierten Minterme evaluiert. Dabei können nur die Blöcke blau-weiß und weiß-grau potentiell zusammenfassbare Mintermpaare umfassen.

	x_3	x_2	x_1	x_0	**OK**
m_8, m_9	1	0	0	-	.
m_8, m_{10}	1	0	-	0	.
m_8, m_{12}	1	-	0	0	.
m_6, m_7	0	1	1	–	.
m_6, m_{14}	–	1	1	0	.
m_9, m_{11}	1	0	–	1	.
m_9, m_{13}	1	–	0	1	.
m_{10}, m_{11}	1	0	1	–	.
m_{10}, m_{14}	1	–	1	0	.
m_{12}, m_{13}	1	1	0	–	.
m_{12}, m_{14}	1	1	–	0	.

Kombinations.py

```
1  import numpy as np
2  minterms = [[0, 1, 1, 0],
3              [0, 1, 1, 1],
4              [1, 0, 0, 0],
5              [1, 0, 0, 1],
6              [1, 0, 1, 0],
7              [1, 0, 1, 1],
8              [1, 1, 0, 0],
9              [1, 1, 0, 1],
10             [1, 1, 1, 0]]
11 distances = np.zeros([len(minterms),len(minterms)]).astype('float')
12 for i in range(0, len(minterms)):
13     for k in range(0, i):
14         diff = np.subtract(minterms[i], minterms[k])
15         dist = np.sum(np.abs(diff))
16         distances[i,k] = dist
17
18 print("Distanzen der Minterme")
19 for j in range(0, len(distances)):  # zeilenweise ausgeben
20     print(distances[j])
21 print("Kombinationen mit Distanz 1: {}".format(np.count_nonzero(dista
     == 1)))
```

```
Distanzen der Minterme
[0. 0. 0. 0. 0. 0. 0. 0. 0.]
[1. 0. 0. 0. 0. 0. 0. 0. 0.]
[3. 4. 0. 0. 0. 0. 0. 0. 0.]
[4. 3. 1. 0. 0. 0. 0. 0. 0.]
[2. 3. 1. 2. 0. 0. 0. 0. 0.]
[3. 2. 2. 1. 1. 0. 0. 0. 0.]
[2. 3. 1. 2. 2. 3. 0. 0. 0.]
[3. 2. 2. 1. 3. 2. 1. 0. 0.]
[1. 2. 2. 3. 1. 2. 1. 2. 0.]
Kombinationen mit Distanz 1: 11
```

Die 11 Paare mit Distanz 1 sind genau die 11 Zeilen der Tabelle oben.

Die zweite Stufe wiederholt die Vorgänge - Sortierung und Evaluation erneut.

	x_3	x_2	x_1	x_0
m_8, m_9	1	0	0	-
m_8, m_{10}	1	0	-	0
m_8, m_{12}	1	-	0	0
m_6, m_7	0	1	1	–
m_6, m_{14}	–	1	1	0
m_9, m_{11}	1	0	–	1
m_9, m_{13}	1	–	0	1
m_{10}, m_{11}	1	0	1	–
m_{10}, m_{14}	1	–	1	0
m_{12}, m_{13}	1	1	0	–
m_{12}, m_{14}	1	1	–	0

Es dürfen nur Zeilen zusammengefasst werden, die die „-“ an den gleichen Positionen haben (!). Außerdem tauchen ab der zweiten Zusammenfassung Terme doppelt auf, welche aber nur einmal notiert werden.

	x_3	x_2	x_1	x_0
m_8, m_9, m_{10}, m_{11}	1	0	-	-
m_8, m_9, m_{12}, m_{13}	1	-	0	-
m_8, m_{10}, m_9, m_{11}	1	0	-	-
$m_8, m_{10}, m_{12}, m_{14}$	1	-	-	0
m_8, m_{12}, m_9, m_{13}	1	-	0	-
$m_8, m_{12}, m_{10}, m_{14}$	1	-	-	0

Eine weitere Zusammenfassung ist nicht möglich. Zwei Terme der ersten Stufe, m_6, m_7 und m_6, m_{14}, ließen sich in der zweiten Stufe mit keinem anderen zusammenfassen.

	x_3	x_2	x_1	x_0
m_8, m_9, m_{10}, m_{11}	1	0	-	-
m_8, m_9, m_{12}, m_{13}	1	-	0	-
m_8, m_{10}, m_9, m_{11}	1	0	-	-
$m_8, m_{10}, m_{12}, m_{14}$	1	-	-	0
m_8, m_{12}, m_9, m_{13}	1	-	0	-
$m_8, m_{12}, m_{10}, m_{14}$	1	-	-	0

Mit diesen beiden Termen ergeben sich insgesamt 5 **Primimplikanten**.

> Ein **Implikant** von f ist ein Produktterm p, für den gilt: Ist $p = 1$, dann ist auch $f = 1$. Ein **Primimplikant** ist ein Implikant, von dem man kein Literal streichen kann, ohne diese Eigenschaft zu verlieren. Im KV-Diagramm ist das eine Schleife, die sich nicht weiter vergrößern lässt.

$$
\begin{aligned}
P_1 = &\quad \{m_8, m_9, m_{10}, m_{11}\} = x_3\bar{x}_2 \\
P_2 = &\quad \{m_8, m_9, m_{12}, m_{13}\} = x_3\bar{x}_1 \\
P_3 = &\quad \{m_8, m_{10}, m_{12}, m_{14}\} = x_3\bar{x}_0 \\
P_4 = &\quad \{m_6, m_7\} = \bar{x}_3x_2x_1 \\
P_5 = &\quad \{m_6, m_{14}\} = x_2x_1\bar{x}_0
\end{aligned}
$$

Darstellung der Minterme im Karnaugh-Diagramm

Im KV-Diagramm sieht man, was die fünf Primimplikanten sind: die größtmöglichen Schleifen. Manche Felder liegen in mehreren Schleifen, wir brauchen also nicht alle.

$$
\begin{aligned}
P_1 = &\quad \{m_8, m_9, m_{10}, m_{11}\} = x_3\bar{x}_2 \\
P_2 = &\quad \{m_8, m_9, m_{12}, m_{13}\} = x_3\bar{x}_1 \\
P_3 = &\quad \{m_8, m_{10}, m_{12}, m_{14}\} = x_3\bar{x}_0 \\
P_4 = &\quad \{m_6, m_7\} = \bar{x}_3x_2x_1 \\
P_5 = &\quad \{m_6, m_{14}\} = x_2x_1\bar{x}_0
\end{aligned}
$$

	$\bar{x}_1\bar{x}_0$	$\bar{x}_1 x_0$	$x_1 x_0$	$x_1\bar{x}_0$
$\bar{x}_3\bar{x}_2$	0	0	0	0
$\bar{x}_3 x_2$	0	0	1 (m_7)	1 (m_6)
$x_3 x_2$	1 (m_{12})	1 (m_{13})	0	1 (m_{14})
$x_3\bar{x}_2$	1 (m_8)	1 (m_9)	1 (m_{11})	1 (m_{10})

Visualisierung der generierten Primimplikanten

	$\bar{x}_1\bar{x}_0$	$\bar{x}_1 x_0$	$x_1 x_0$	$x_1\bar{x}_0$
$\bar{x}_3\bar{x}_2$	0	0	0	0
$\bar{x}_3 x_2$	0	0	1 (P_4)	1 (P_4, P_5)
$x_3 x_2$	1 (P_2, P_3)	1 (P_2)	0	1 (P_3, P_5)
$x_3\bar{x}_2$	1 (P_1, P_2, P_3)	1 (P_1, P_2)	1 (P_1)	1 (P_1, P_3)

Die meisten Minterme werden mehrfach abgedeckt, nur m_7, m_{11} und m_{13} von genau einem Primimplikanten. Wir müssen also nicht alle fünf verwenden.

Die **Primimplikantentafel**, auch zweite Quine'sche Tabelle genannt, zeigt, welcher Primimplikant welche Minterme abdeckt.

	m_{14}	m_{13}	m_{12}	m_{11}	m_{10}	m_9	m_8	m_7	m_6
P_1				x	x	x	x		
P_2		x	x			x	x		
P_3	x		x		x		x		
P_4								x	x
P_5	x								x

Welche Primimplikanten müssen in jeder Lösung vorkommen?

- [ ] P_1
- [ ] P_2
- [ ] P_3
- [ ] P_4
- [ ] P_5

Lösung der minimalen Überdeckung - intuitiv

Ein Primimplikant, der als einziger einen Minterm abdeckt, heißt essentiell. Er gehört in jede Lösung. Danach bleibt meist nur noch wenig zu entscheiden.

Die Minterme m_{13}, m_{11} und m_7 kommen nun in jeweils einem Primimplikanten vor. Folglich sind P_1, P_2 und P_4 sogenannte **essentielle Primimplikanten**.

	m_{14}	m_{13}	m_{12}	m_{11}	m_{10}	m_9	m_8	m_7	m_6
P_1				x	x	x	x		
P_2		x	x			x	x		
P_3	x		x		x		x		
P_4								x	x
P_5	x								x

	$\bar{x}_1\bar{x}_0$	$\bar{x}_1x_0$	x_1x_0	$x_1\bar{x}_0$
$\bar{x}_3\bar{x}_2$	0	0	0	0
$\bar{x}_3x_2$	0	0	m_7 (P_4)	m_6 (P_4, P_5)
x_3x_2	m_{12} (P_2, P_3)	m_{13} (P_2)	0	m_{14} (P_3, P_5)
$x_3\bar{x}_2$	m_8 (P_1, P_2, P_3)	m_9 (P_1, P_2)	m_{11} (P_1)	m_{10} (P_1, P_3)

Aussage 1: P_2 ist essentiell, weil nur er m_{13} abdeckt. Mit P_2 sind auch m_{12}, m_9 und m_8 abgedeckt, diese Spalten entfallen.

	m_{14}	m_{13}	m_{11}	m_{10}	m_7	m_6
P_1			x	x		
P_2		x				
P_3	x			x		
P_4					x	x
P_5	x					x

Aussage 2: P_1 ist essentiell wegen m_{11} und deckt auch m_{10} ab.

Aussage 3: P_4 ist essentiell wegen m_7 und deckt auch m_6 ab.

Nach den drei essentiellen Primimplikanten ist nur noch m_{14} offen:

	m_{14}
P_3	x
P_5	x

P_3 und P_5 decken beide den noch offenen Minterm m_{14} ab. Welcher sollte gewählt werden?

$$
\begin{aligned}
P_1 &= \{m_8, m_9, m_{10}, m_{11}\} = x_3\bar{x}_2 \\
P_2 &= \{m_8, m_9, m_{12}, m_{13}\} = x_3\bar{x}_1 \\
P_3 &= \{m_8, m_{10}, m_{12}, m_{14}\} = x_3\bar{x}_0 \\
P_4 &= \{m_6, m_7\} = \bar{x}_3x_2x_1 \\
P_5 &= \{m_6, m_{14}\} = x_2x_1\bar{x}_0
\end{aligned}
$$

$P_3 = x_3\bar{x}_0$ hat zwei Literale, $P_5 = x_2x_1\bar{x}_0$ drei. Wir wählen P_3.

Ergebnis: Unsere relevanten Primimplikanten sind
$y = P_1 + P_2 + P_3 + P_4 = x_3\bar{x}_2 + x_3\bar{x}_1 + x_3\bar{x}_0 + \bar{x}_3x_2x_1$.

Lösung der minimalen Überdeckung - algorithmisch

Bei großen Tafeln hilft ein festes Vorgehen. Die Spaltendominanz streicht Minterme, die automatisch mit abgedeckt werden.

Spaltendominanzprüfung

Jede Spalte steht für einen Minterm, ihre Kreuze zeigen, welche Primimplikanten ihn abdecken. Wir lesen die Spalte deshalb als Menge von Primimplikanten. Eine Spalte A **dominiert** eine Spalte B, wenn $A \supseteq B$ gilt, wenn also jeder Primimplikant aus B auch in A vorkommt.

Regel: Die dominierende Spalte wird gestrichen. Jede Auswahl, die B abdeckt, deckt automatisch auch A ab.

	m_1	m_2	m_3
P_A	x	x	
P_B	x		x
Prüfung	streichen		

Hier ist $m_1 = \{P_A, P_B\}$, $m_2 = \{P_A\}$ und $m_3 = \{P_B\}$. Weil $m_1 \supseteq m_2$ gilt, dominiert m_1 und wird gestrichen: Wer m_2 abdeckt, wählt P_A und deckt damit auch m_1 ab.

An der Kaffeemaschine: $m_{13} = \{P_2\}$ ist in m_8, m_9 und m_{12} enthalten, $m_{11} = \{P_1\}$ in m_{10} und $m_7 = \{P_4\}$ in m_6. Die Spalten m_6, m_8, m_9, m_{10} und m_{12} entfallen. Übrig bleiben m_7, m_{11}, m_{13} und m_{14}.

Die Zeilendominanz streicht Primimplikanten, die ein anderer vollständig ersetzt. Danach wiederholt man beide Prüfungen, bis sich nichts mehr ändert.

Zeilendominanzprüfung

Jede Zeile steht für einen Primimplikanten, ihre Kreuze zeigen, welche Minterme er abdeckt. Eine Zeile A **dominiert** eine Zeile B, wenn $A \supseteq B$ gilt, wenn also A alle Minterme von B abdeckt.

Regel: Die dominierte Zeile wird gestrichen, sofern die dominierende nicht mehr Literale hat. Sonst wäre die Zeile mit der Obermenge zwar vollständiger, aber teurer.

	m_1	m_2	m_3	**Prüfung**
P_A	x	x		streichen
P_B	x	x	x	

Hier ist $P_A = \{m_1, m_2\}$ und $P_B = \{m_1, m_2, m_3\}$. Weil $P_B \supseteq P_A$ gilt, dominiert P_B, und P_A wird gestrichen.

Achtung: Bei Spalten fällt die **dominierende** weg, bei Zeilen die **dominierte**.

An der Kaffeemaschine: In der Resttafel decken P_3 und P_5 beide nur noch m_{14} ab, sie dominieren sich also gegenseitig. P_3 hat zwei Literale, P_5 drei, deshalb fällt P_5 weg. Jetzt ist jede Spalte genau einmal abgedeckt, und es bleiben P_1 bis P_4.

Ablauf der Überdeckung:

1. Essentielle Primimplikanten übernehmen und ihre Spalten streichen.
2. Spaltendominanz anwenden.
3. Zeilendominanz anwenden.
4. Schritte 1 bis 3 wiederholen, bis sich nichts mehr ändert.
5. Bleibt eine Tafel ohne essentielle Primimplikanten und ohne Dominanz übrig, heißt sie **zyklisch**. Dann wählt man probeweise einen Primimplikanten, rechnet weiter und vergleicht die Ergebnisse. Ein Beispiel dafür ist Aufgabe 1 der Hausaufgabe.

Die minimale Überdeckung ist allgemein ein schweres kombinatorisches Problem. Werkzeuge für große Schaltungen arbeiten deshalb mit Heuristiken, die eine gute, aber nicht immer die minimale Lösung finden.

Für das Verfahren von Quine-McCluskey existieren webbasierte Werkzeuge. Vollziehen Sie den Ablauf zum Beispiel mit den Werkzeugen der TU Ilmenau nach: https://goldi-labs.net/SANE/

Schaltungssynthese

Jetzt wird aus der Formel eine Schaltung. Ein Schaltnetz besteht aus Gattern ohne Rückkopplung, und jede Schaltfunktion lässt sich in zwei Stufen umsetzen.

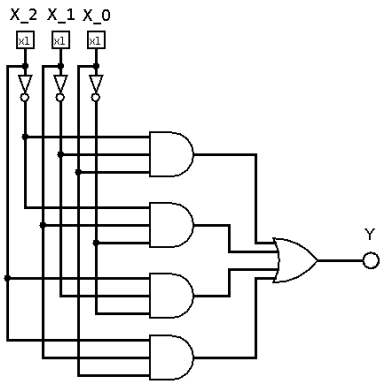
> **Schaltungssynthese** beschreibt die Umsetzung einer booleschen Funktion in eine Hardware-Schaltung. Grundlage sind Logikgatter, die als spezifische Schaltnetze industriell gefertigt werden.

Beispiel

$y = f(x_0, x_1, x_2)$

Die Funktion evaluiert die Parität der Variablen. Für eine ungerade Zahl von Einsen wird eine „1“ ausgegeben, sonst eine „0“.

x_2	x_1	x_0	y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



> Ein Schaltnetz ist eine schaltungstechnische Realisierung einer Schaltfunktion $f: \{0,1\}^n \to \{0,1\}^m$ mit $n, m \geq 1$. f ist zerlegbar in m Boolesche Funktionen mit den gleichen n Eingangsvariablen: $f_1(x_1, x_2, ..., x_n), f_2(x_1, x_2, ..., x_n), ..., f_m(x_1, x_2, ..., x_n)$

Jedes Schaltnetz ist als gerichteter, azyklischer Graph darstellbar:

- Gatter, Ein- und Ausgänge sind die Knoten
- Verbindungsleitungen entsprechen den gerichteten Kanten
- Zyklen (Rückkopplungen) sind nicht zulässig!

Aus der Darstellung als kanonische Normalform resultiert, dass jede Schaltfunktion durch ein zweistufiges Schaltnetz realisierbar ist, wenn

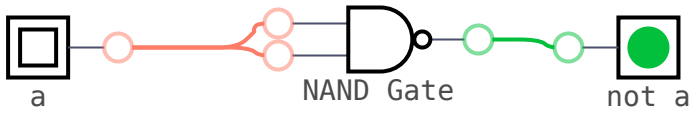
- alle Eingangssignale sowohl einfach als auch negiert vorliegen
- Gatter mit einer ausreichenden Größe (d.h. Anzahl an Eingängen)

zur Verfügung stehen

> **Merke:**
>
> - Jedes Schaltnetz lässt sich allein aus UND-, ODER- und NICHT-Gattern aufbauen.
> - Jedes Schaltnetz lässt sich allein aus NAND-Gattern aufbauen.
> - Jedes Schaltnetz lässt sich allein aus NOR-Gattern aufbauen.
>
> Wie das mit NAND geht, zeigen die drei Schaltungen darunter.

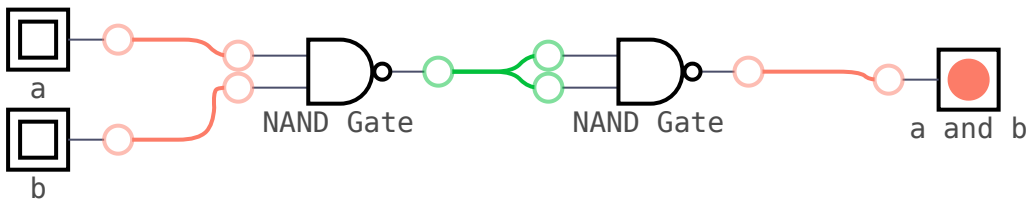
NOT

$\bar{a} = a \barwedge a$



AND

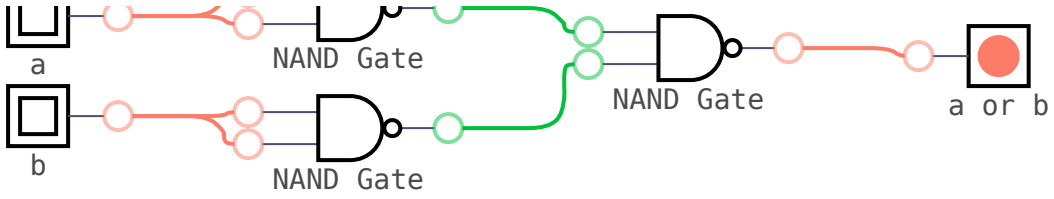
$a \cdot b = a \barwedge b$



OR

$a + b = \bar{a} \barwedge \bar{b}$



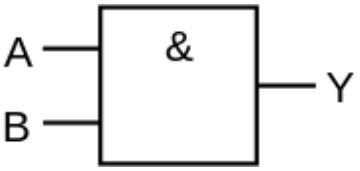
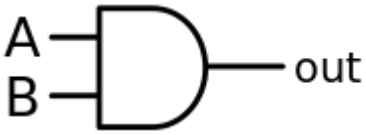
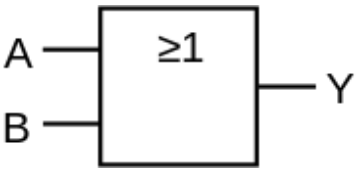
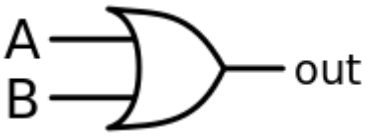
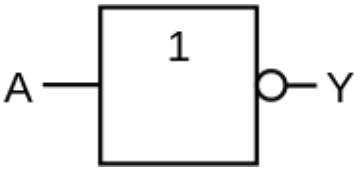
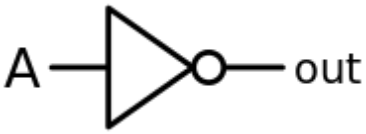
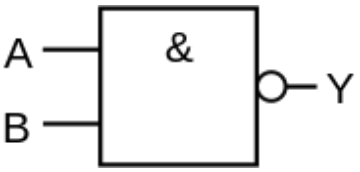
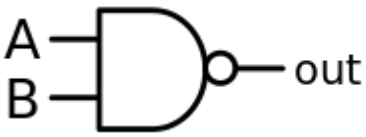
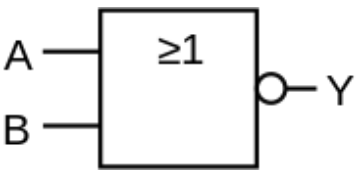
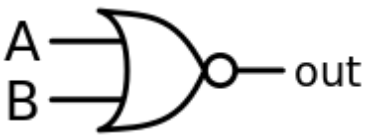
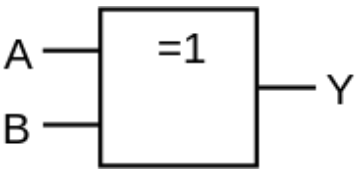
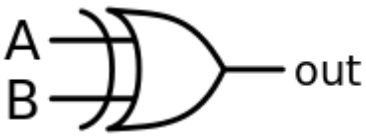
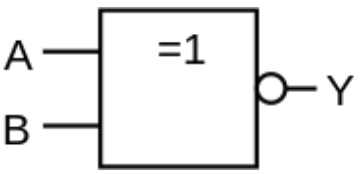
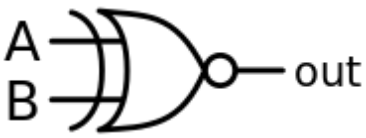


Logikgatter als Grundlage der Schaltnetze

Für dieselben Gatter gibt es zwei Zeichennormen. In deutschen Datenblättern finden Sie oft die eckigen IEC-Symbole, in englischsprachigen fast immer die runden amerikanischen.

Ein Logikgatter, auch nur Gatter ist die technische Realisierung einer booleschen Funktion, die binäre Eingangssignale zu einem binären Ausgangssignal verarbeitet. Die Eingangssignale werden durch Implementierung logischer Operatoren, wie der Konjunktion (Und-Gatter), der Disjunktion (Oder-Gatter), der Kontravalenz (Exklusiv-Oder-Gatter) oder der Negation (Nicht-Gatter) zu einem einzigen logischen Ergebnis umgewandelt und auf das Ausgangssignal abgebildet.

Gatterfunktionen können zudem einen negierten Ausgang abbilden: NAND-Gatter (Nicht-Und), NOR-Gatter (Nicht-Oder), XNOR-Gatter (Nicht-Exklusiv-Oder).

Funktion	IEC 60617-12 : 1997	ANSI/IEEE Std 91/91a-1991
$UN \quad D$ $Y \quad = A \cdot B$ $Y \quad = A \wedge B$		
OR $Y \quad = A + B$ $Y \quad = A \vee B$		
$NO \quad T$ $Y \quad = \bar{A}$ $Y \quad = \neg A$		
$NA \quad ND$ $Y \quad = A \barwedge B$ $Y \quad = A \bar{\wedge} B = A \barwedge B$		
$NO \quad R$ $Y \quad = A \barvee B$ $Y \quad = A \bar{\vee} B = A \barvee B$		
$XO \quad R$ $Y \quad = A \oplus B$ $Y \quad = A \underline{\vee} B$		
$XN \quad OR$ $Y \quad = A \odot B$ $Y \quad = A \underline{\bar{\vee}} B = A \underline{\barvee} B$		

Im englischen Sprachraum waren und sind die amerikanischen Symbole (rechte Spalte) üblich. Die IEC-Symbole sind international auf beschränkte Akzeptanz gestoßen und werden in der amerikanischen Literatur (fast) durchgängig ignoriert. In der Automatisierungstechnik sind sie dagegen Standard: Der Funktionsplan (FUP) zur Programmierung von Steuerungen nach IEC 61131-3 verwendet die Kästchen mit `&` und `≥1`.

Zweistufige Schaltungssynthese

Zweistufig heißt: erst eine Ebene UND-Gatter, dann ein ODER-Gatter. Die Schaltung darunter setzt die KDNF des Beispiels genau so um.

Dazu werden für eine KDNF je Minterm ein UND-Gatter und ein ODER-Gatter zur Disjunktion aller Minterme benötigt. Für die KKNF sind es entsprechend je Maxterm ein ODER-Gatter und ein UND-Gatter zur Konjunktion aller Maxterme.

Die Anzahl der benötigten Gatter zur Realisierung der KDNF (bzw. KKNF) einer n-stelligen Schaltfunktion:

- je Boolescher Funktion max. 2^n UND-Gatter (bzw. ODER-Gatter) mit jeweils max. n Eingängen
- je Boolescher Funktion ein ODER-Gatter (bzw. UND-Gatter) mit max. 2^n Eingängen

Beispiel:

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

Die Schaltung setzt die KDNF $Y = \bar{A}\bar{B}\bar{C} + \bar{A}BC + A\bar{B}\bar{C}$ um. Der Simulator kennt nur Gatter mit zwei Eingängen, deshalb besteht jedes UND mit drei Eingängen aus zwei Gattern. Minimiert wäre $Y = \bar{B}\bar{C} + \bar{A}BC$. Prüfen Sie das im Diagramm:

Felder eintragen | Schleifen bilden | DNF (Einsen) | KNF (Nullen)

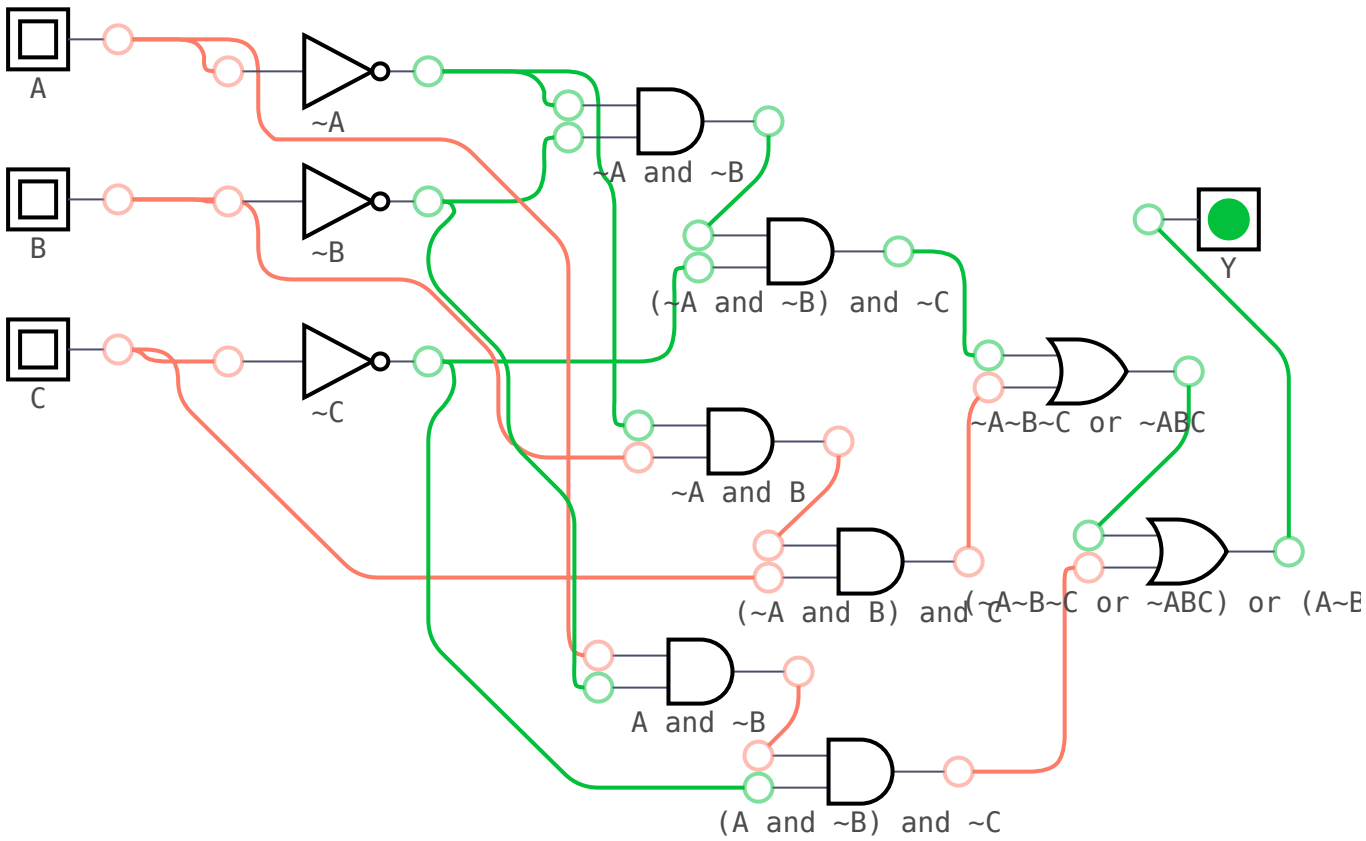
$\overline{C}$	$\overline{A}\,\overline{B}$	$\overline{A}B$	AB	$A\overline{B}$
	1 (0)	0 (2)	0 (6)	1 (4)
C	0 (1)	1 (3)	0 (7)	0 (5)

Schleife übernehmen | Lösung zeigen | Zurücksetzen

Felder anklicken, die eine Schleife bilden.

$f = ?$

Noch nicht überdeckt: 3 Einsen.

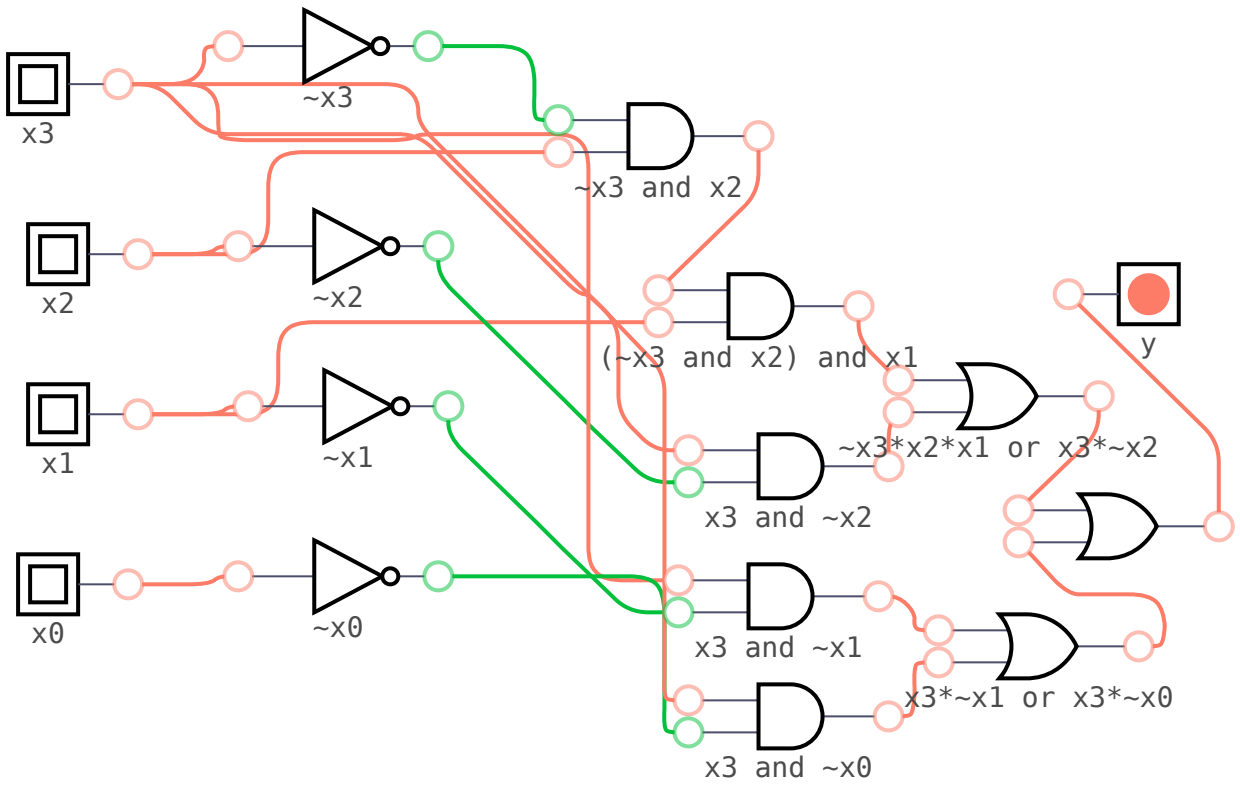


Zurück zum Beispiel - Schritt 3 Umsetzung

Zurück zur Kaffeemaschine. Probieren Sie die Schaltung aus: Die Lampe leuchtet genau in den Zuständen 6 bis 14.

Für unser Kaffeemaschinenbeispiel kann eine Lösung also zum Beispiel folgendermaßen aussehen. Wir hatten ja folgende Funktion als minimale Darstellung identifiziert:

$$y = \bar{x}_3 x_2 x_1 + x_3 \bar{x}_2 + x_3 \bar{x}_1 + x_3 \bar{x}_0$$



Für die Zustände 6 bis 14 soll ein Fehler ausgegeben werden:

x_3	x_2	x_1	x_0	y
0	0	0	0	0
…	…	…	…	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

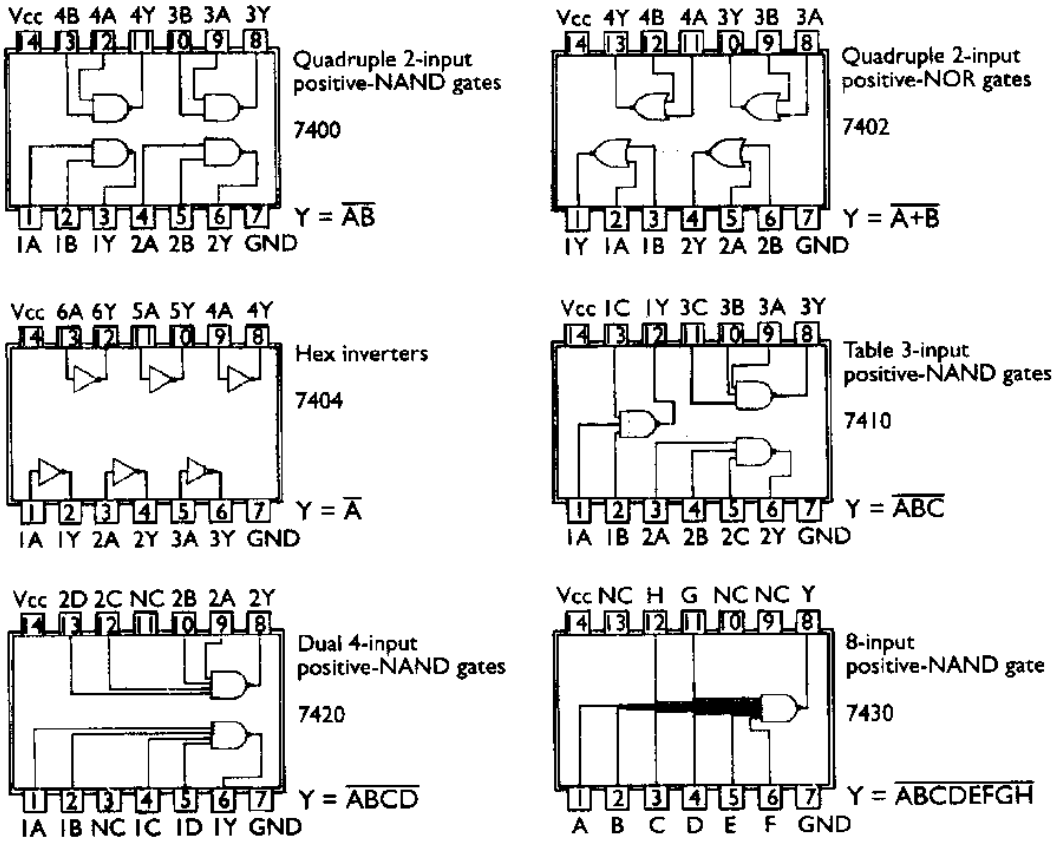
Worin unterscheidet sich unsere minimale Form von der technischen Umsetzung? Die Formel braucht ein ODER mit vier Eingängen. Der Simulator kennt nur Gatter mit zwei Eingängen, deshalb wird daraus ein Baum aus drei ODER-Gattern in zwei Stufen. Das Signal läuft also durch mehr Gatter, als die zweistufige Form vermuten lässt.

Spielen Sie ein wenig mit der Simulation, um sich von der Wirksamkeit unserer Lösung zu überzeugen.

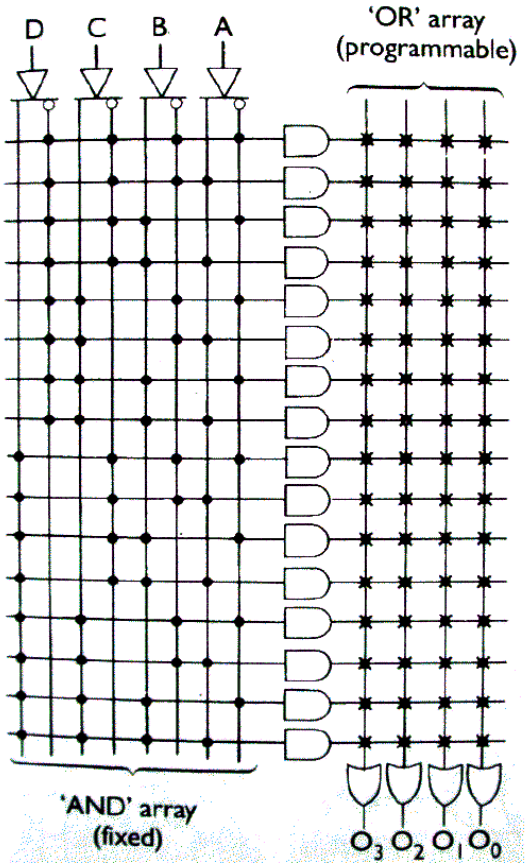
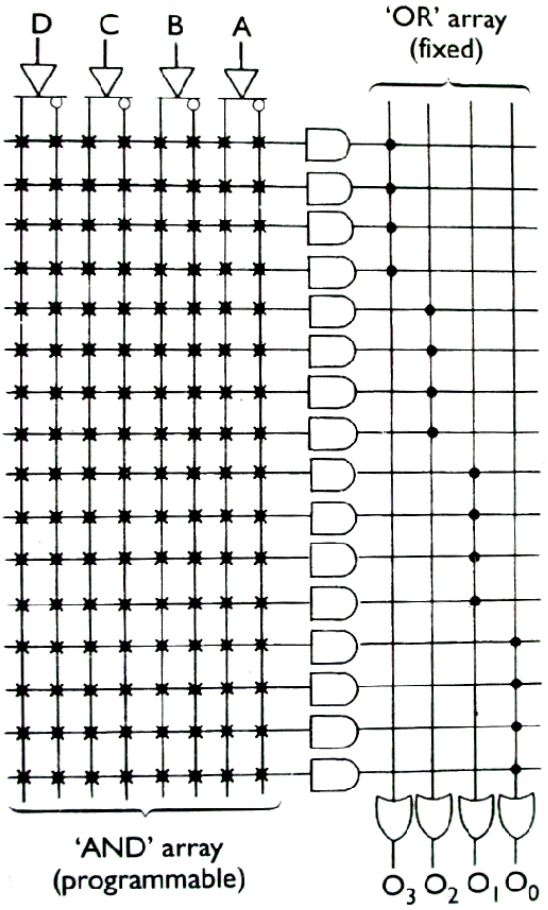
Umsetzung

Für einfache Schaltnetze gibt es fertige Gatterbausteine. Größere Schaltnetze baut man aus programmierbaren Bausteinen mit einer UND-Ebene und einer ODER-Ebene, also genau der zweistufigen Form. PROM und PAL unterscheiden sich darin, welche Ebene man programmiert. Das entscheidet, ob man minimieren muss.

Prinzipiell lassen sich alle logischen Verknüpfungen als Gatter realisieren. Die 74xx-Reihe umfasst dabei die Basisbausteine und darüber hinaus integrierte Schaltnetze, die höhere Funktionen bereitstellen (Addierwerk, Multiplexer, Decoder), siehe die Liste der 74xx-Familie.



Aufwändigere Schaltnetze baut man aus programmierbaren Bausteinen. Sie bestehen aus einer UND-Matrix und einer ODER-Matrix, also genau der zweistufigen Form von oben. Sie unterscheiden sich darin, welche der beiden Matrizen programmierbar ist.

PROM - „Programmable Read-Only Memory“	PAL - „Programmable Array Logic“
16 Worte zu 4 Bit	4 Ein- und Ausgänge und 4 Produktterme je Ausgang
	
Adressdekoder entspricht einer festen UND-Matrix, Koppelelemente bilden eine programmierbare ODER-Matrix	programmierbare UND-Matrix, Produktterme werden mit fester ODER-Matrix verknüpft

So lesen Sie die Bilder: Ein Punkt ist eine feste Verbindung, ein Kreuz eine programmierbare. Jeder Eingang liegt einfach und, über den Kreis am Puffer, negiert vor. Die waagerechte Linie vor jedem UND-Gatter steht für alle seine Eingänge zugleich, ein UND-Gatter verknüpft also alle Signale, mit denen seine Linie verbunden ist.

PROM: Die UND-Matrix ist fest verdrahtet. Für die vier Eingänge D, C, B, A erzeugt sie alle $2^4 = 16$ Minterme, jede Zeile im Bild ist ein Minterm. Programmiert wird nur die ODER-Matrix: An jedem Kreuz legt man fest, ob dieser Minterm in die Summe eines Ausgangs eingeht. Für jeden Ausgang trägt man also seine KDNF ein, und das ist nichts anderes als seine Spalte der Wahrheitstafel. Deshalb heißt

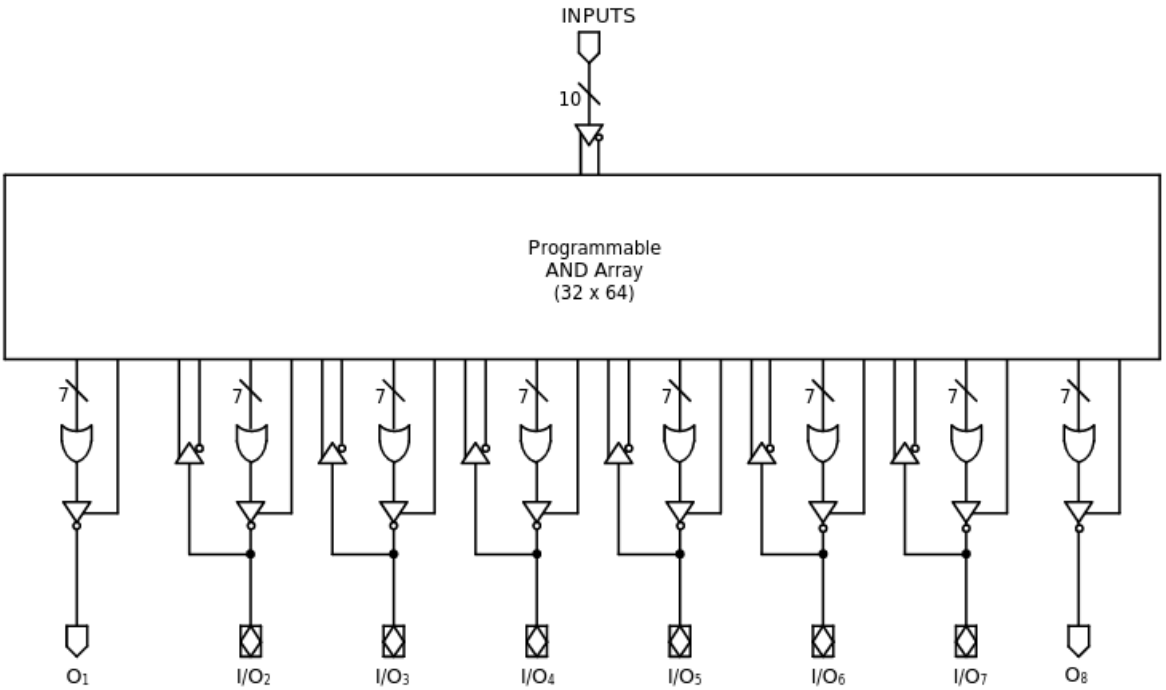
der Baustein Speicher: Die Eingänge bilden die Adresse, die Ausgänge das gespeicherte Wort. Im Controller entspricht dem eine Wertetabelle im Programmspeicher, wie die Tabelle `d_tab` für Segment `d` aus Vorlesung 03.

> Ein PROM mit 2^n Worten zu je m Bit realisiert jede Schaltfunktion $f\colon \{0,1\}^n \to \{0,1\}^m$, ganz ohne Minimierung.

Für die Kaffeemaschine legt man x_3, x_2, x_1, x_0 an die Eingänge D, C, B, A und nutzt O_0 als y. In die Spalte O_0 kommen Kreuze in den Zeilen der Minterme 6 bis 14.

PAL: Hier ist es umgekehrt. Die UND-Matrix ist programmierbar, jedes UND-Gatter bildet einen frei wählbaren Produktterm aus den Eingängen und ihren Negationen. Die ODER-Matrix ist fest: Im Bild bekommt jeder Ausgang genau 4 Produktterme. Ein PAL realisiert also jede DNF, solange die Zahl der Terme je Ausgang reicht. Hier zählt die Minimierung.

GAL („Generic Array Logic“) ist ein PAL, das sich elektrisch löschen und neu programmieren lässt. Seine Ausgänge haben konfigurierbare Ausgangszellen, die weiter unten bei den Glitches noch eine Rolle spielen. Reale Bausteine haben deutlich mehr Eingänge und Terme als das Schema oben. Heute übernehmen CPLDs und FPGAs diese Rolle in größerem Maßstab.



Aus dem Datenblatt der PAL16R8-Familie, zu der der PAL16L8 gehört, Advanced Micro Devices [1]

	PROM	PAL
UND-Matrix	fest, erzeugt alle Minterme	programmierbar
ODER-Matrix	programmierbar	fest, begrenzte Zahl von Termen je Ausgang
Minimierung nötig?	nein, man speichert die Wahrheitstafel	ja, wenn die Terme sonst nicht reichen
günstig, wenn	die Funktion kaum Struktur hat und minimiert immer noch viele Terme bräuchte; der Aufwand wächst mit 2^n	viele Variablen und relativ wenige Terme vorkommen, viele Eingangsbelegungen auf dieselbe Ausgangsbelegung führen

Die Kaffeemaschine soll in das PAL aus der Abbildung, mit 4 Produkttermen je Ausgang. Welche Form von y passt?

- ◯ die KDNF mit 9 Mintermen
- ◯ die minimierte Form mit 4 Termen
- ◯ beide
- ◯ keine von beiden

[1] Datenblatt PAL16R8 Family, Advanced Micro Devices, link, 1996

Herausforderungen

Bisher haben wir so getan, als schalteten Gatter sofort. In Wirklichkeit braucht jedes Gatter einige Nanosekunden, und die Zeiten sind nie ganz gleich. Dadurch kann der Ausgang kurz einen falschen Wert zeigen, obwohl die Formel stimmt.

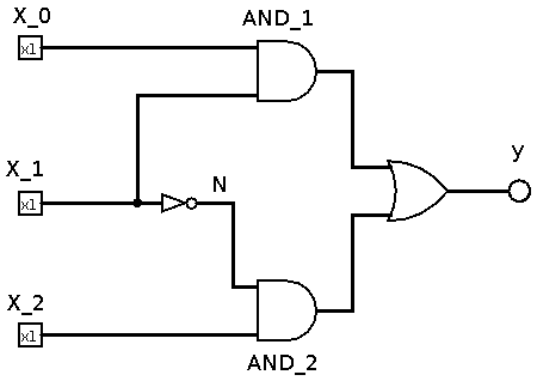
Als **Glitch** bezeichnet man einen kurzzeitig falschen Wert am Ausgang einer logischen Schaltung. In der Literatur heißt das **Hazard**, ein kurzer Einbruch eines Ausgangs, der eigentlich auf 1 bleiben sollte, genauer **statischer 1-Hazard**. Er tritt auf, weil Signale auf verschiedenen Wegen unterschiedlich lange brauchen, im Beispiel unten wegen eines zusätzlichen Inverters auf einem der Wege.

Ausgangspunkt

Jedes Gatter braucht Zeit. Beim UND-Gatter aus der Hausaufgabe zu Vorlesung 02 sind es bei 3,3 V bis zu 5,7 ns (Datenblatt SN74LVC2G08).

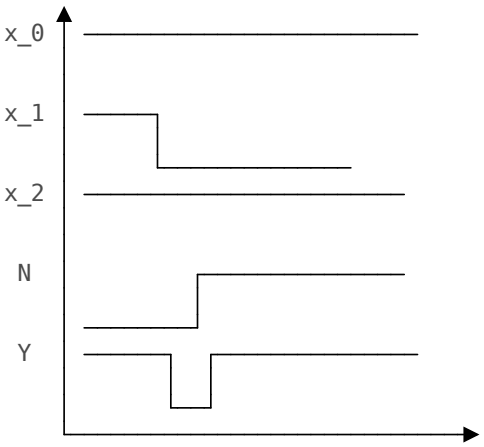
Konsequenz

Schaltnetz für die Umsetzung der Schaltfunktion $y = x_0 x_1 + \bar{x}_1 x_2$



Die Schaltung befindet sich zunächst in der Konfiguration $x_0 = x_1 = x_2 = 1$. Entsprechend liegt für y ebenfalls eine 1 vor.

Nun wechselt x_1 auf 0. Das UND-Gatter für $x_0 x_1$ geht sofort auf 0. Der Inverter braucht aber eine gewisse Zeit, um seinen Ausgang N von 0 auf 1 umzuschalten. Für kurze Zeit ist sowohl $x_1 = 0$ als auch $N = 0$. Entsprechend schaltet auch „AND_2“ noch nicht durch, beide UND-Gatter liefern 0, und y fällt kurz auf 0.



Bei einer Fehler-LED sieht niemand einen Einbruch von wenigen Nanosekunden. Gefährlich wird ein Glitch dort, wo ein kurzer Puls als Ereignis zählt: an einem Takt- oder Zähleingang, an einem Interrupt-Eingang eines Controllers oder an einem Reset.

Analyse im Karnaugh-Veitch-Diagramm

Das potentielle Auftreten eines Glitches wird im Karnaugh-Veitch Diagramm anhand nebeneinander liegender, nicht durch Überlappungen verbundener Schleifen deutlich.

	$\bar{x}_1\bar{x}_0$	$\bar{x}_1 x_0$	$x_1 x_0$	$x_1\bar{x}_0$
$\bar{x}_2$	0	0	1	0
x_2	1	1	1	0

Zwei Auswege gibt es. Entweder man fügt einen Term hinzu, der den Übergang überbrückt, auch wenn er die Formel länger macht. Oder man liest den Ausgang erst, wenn alle Signale eingeschwungen sind.

Lösungsansätze

1. Integration glitchfreier Pfade, indem wir die Überlappung im Karnaugh-Veitch-Diagramm über den Minimierungsgedanken stellen. Der zusätzliche Term x_0x_2 hängt nicht von x_1 ab. Er hält y auf 1, während x_1 wechselt. Algebraisch ist er der **Konsensterm** von x_0x_1 und $\bar{x}_1x_2$. Er hilft, wenn sich nur eine Eingangsvariable ändert. Das KV-Diagramm meldet ihn als „überflüssig“, für das Zeitverhalten ist er aber nötig.

$y = x_0x_1 + \bar{x}_1x_2 + x_0x_2$

Probieren Sie es aus: Bilden Sie im Diagramm zuerst die beiden Schleifen der minimalen Form, dann zusätzlich die Schleife für x_0x_2. Das Diagramm meldet sie als überflüssig. Für die Logik stimmt das, für das Zeitverhalten nicht.

Felder eintragen | Schleifen bilden | DNF (Einsen) | KNF (Nullen)

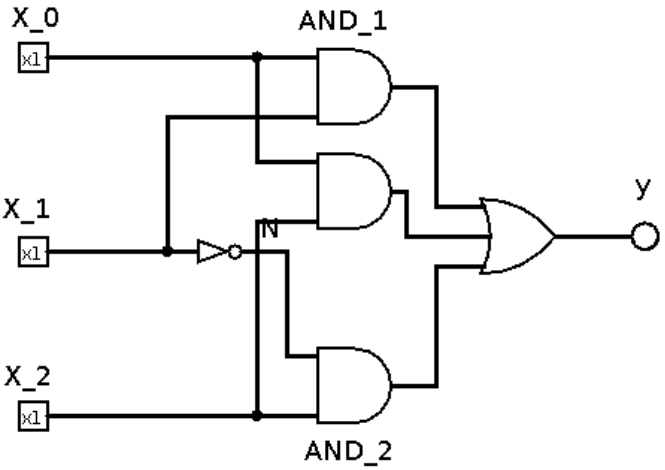
	$\bar{x}_2\bar{x}_1$	$\bar{x}_2 x1$	x2x1	$x2\bar{x}_1$
$\bar{x}_0$	0 (0)	0 (2)	0 (6)	1 (4)
x0	0 (1)	1 (3)	1 (7)	1 (5)

Schleife übernehmen | Lösung zeigen | Zurücksetzen

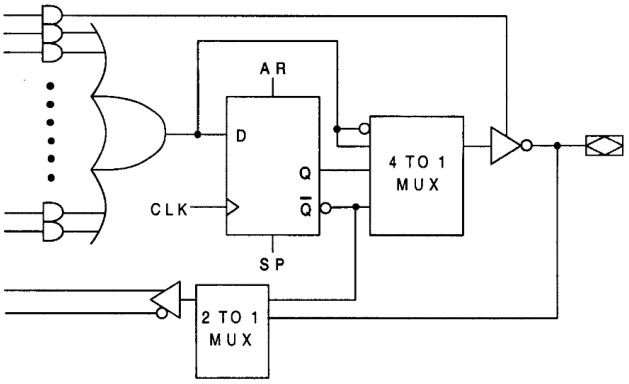
Felder anklicken, die eine Schleife bilden.

f = ?

Noch nicht überdeckt: 4 Einsen.



2. Synchronisation gültiger Zustände: Das Bild zeigt eine Ausgangszelle eines GAL. Ein Multiplexer wählt, ob der Ausgang direkt aus dem ODER kommt oder über ein D-Flipflop. Das Flipflop übernimmt den Wert nur zur Taktflanke. Kommt die Flanke erst, wenn alle Signale eingeschwungen sind, ist der Glitch vorbei, bevor er weitergegeben wird. Das ist auch der einzige Ausweg, wenn sich mehrere Eingänge gleichzeitig ändern. Wechselt die Kaffeemaschine von Zustand 7 (0111) auf Zustand 8 (1000), ist y vorher und nachher 1. Kippen die vier Bits aber nicht exakt gleichzeitig, kann kurz zum Beispiel 0000 oder 1111 anliegen, und dort ist $y = 0$. Kein zusätzlicher Term verhindert das. Flipflops lernen Sie in Vorlesung 06 kennen.



Und im ATmega328P?

Der Mehrbit-Fall aus dem Glitch-Abschnitt tritt auch in Software auf. Liest ein Programm die vier Zustandsleitungen der Kaffeemaschine nacheinander, kann es einen Zustand sehen, den es nie gab. Die Lösung ist dieselbe wie in Hardware: alle Bits zum selben Zeitpunkt übernehmen.

Die vier Zustandsleitungen x_3, x_2, x_1, x_0 der Kaffeemaschine hängen an den Pins PD3 bis PD0 des ATmega328P. Zwei Arten, den Zustand zu lesen:

Zustand.c

```c
#include <avr/io.h>
#include <stdint.h>

// Zustand x3..x0 der Kaffeemaschine an PD3..PD0
uint8_t zustand_einzeln(void) {
  uint8_t z = 0;
  if (PIND & (1 << PD3)) z |= 8;
  if (PIND & (1 << PD2)) z |= 4;
  if (PIND & (1 << PD1)) z |= 2;
  if (PIND & (1 << PD0)) z |= 1;
  return z;
}

uint8_t zustand_auf_einmal(void) {
  return PIND & 0x0F;
}
```

Nachgesehen im Maschinencode (`avr-gcc -Os`): `zustand_einzeln` greift viermal nacheinander auf das Eingangsregister `PIND` zu, `zustand_auf_einmal` liest es mit einem einzigen Befehl (`in r24, PIND`).

- **Nacheinander lesen ist ein Glitch in Software.** Wechselt die Maschine zwischen dem ersten und den übrigen Zugriffen von Zustand 7 (`0111`) auf Zustand 8 (`1000`), liest `zustand_einzeln` Bit 3 noch als 0 und die übrigen Bits schon als 0. Das Ergebnis ist 0, ein Zustand, der nie anlag, und die Fehler-LED bliebe aus. Wie beim Mehrbit-Wechsel im Schaltnetz hilft keine geschicktere Formel.
- **Auf einmal lesen ist die Synchronisation.** Ein einziger Lesebefehl liefert alle acht Bits des Ports, übernommen mit derselben Taktflanke. Das Ergebnis ist 7 oder 8, nichts dazwischen. Ganz sicher ist auch das nur, wenn die Leitungen beim Lesen stabil sind, denn ein kurzer Übergang kann auch einen einzelnen Lesebefehl treffen. Deshalb liest man in solchen Fällen zweimal und übernimmt den Wert erst, wenn beide Lesungen übereinstimmen.
- **Der Controller synchronisiert jeden Pin selbst.** Laut Datenblatt bilden das Register `PINx` und ein vorgeschalteter Latch einen Synchronisierer. Er verhindert, dass ein Pin, der genau an der Taktflanke wechselt, einen unbestimmten Wert liefert, verzögert das Signal aber um ½ bis 1½ Taktperioden.[ATmega] Davor sitzt ein Schmitt-Trigger, der verrauschte Flanken in saubere Pegelwechsel verwandelt. Latch und Flipflop lernen Sie in Vorlesung 06 kennen.
- **Wo ein Glitch im Controller schadet:** an einem Interrupt-Eingang. Bei `INT0` und `INT1` lösen Pulse, die länger als eine Taktperiode dauern, sicher einen Interrupt aus, bei kürzeren ist das nicht sichergestellt.[ATmega] Bei 16 MHz dauert eine Taktperiode 62,5 ns. Ein Glitch von wenigen Nanosekunden kann also einen Interrupt auslösen oder auch nicht, und genau solche Fehler sind schwer zu finden.

[ATmega] Microchip Technology: ATmega48A/PA/88A/PA/168A/PA/328/P megaAVR® Data Sheet, DS40002061B, 2020, Abschnitt 14.2.4 „Reading the Pin Value“, S. 86, und Beschreibung der Bits ISC01/ISC00 und ISC11/ISC10 im Register EICRA, S. 80, Link

Was Sie heute mitnehmen

Diese Punkte fassen die Vorlesung zusammen. Die Fragen darunter gehören zu den Lernzielen vom Anfang: Wenn Sie sie beantworten und das Verfahren von Quine-McCluskey an einem eigenen Beispiel durchrechnen können, sind Sie gut auf die Standardschaltnetze in der nächsten Vorlesung vorbereitet.

- **Quine-McCluskey** minimiert eine KDNF algorithmisch: Minterme nach der Zahl ihrer Einsen sortieren, Terme, die sich in genau einer Variablen unterscheiden, stufenweise zusammenfassen – übrig bleiben die **Primimplikanten**.
- **Die Primimplikantentafel** zeigt, welcher Primimplikant welche Minterme abdeckt. **Essentielle Primimplikanten** decken einen Minterm als einzige ab und gehören immer zur Lösung; den Rest reduzieren Spalten- und Zeilendominanz. Bleibt eine zyklische Tafel übrig, probiert man einen Primimplikanten aus.
- **Ein Schaltnetz** setzt eine Schaltfunktion als Schaltung ohne Rückkopplung um. Jede Schaltfunktion lässt sich zweistufig realisieren – und jedes Schaltnetz allein aus NAND- oder allein aus NOR-Gattern aufbauen.
- **PROM und PAL:** Ein PROM bildet die Wahrheitstafel unmittelbar ab und braucht keine Minimierung; ein PAL realisiert eine (minimierte) DNF über eine programmierbare UND-Matrix.
- **Glitches** (Hazards) entstehen, weil Signale auf verschiedenen Wegen unterschiedlich lange brauchen. Im Karnaugh-Veitch-Diagramm erkennt man sie an benachbarten Schleifen ohne Überlappung; ein zusätzlicher, überlappender Term (Konsensterm) beseitigt sie, wenn sich nur eine Variable ändert, sonst hilft nur die Synchronisation gültiger Zustände.

Prüfen Sie sich:

- Erläutern Sie das Verfahren von Quine-McCluskey. Was ist ein Primimplikant, und wann ist er essentiell?
- Grenzen Sie die Begriffe Schaltnetz und Schaltfunktion voneinander ab.
- Welche Gatterkombinationen sind geeignet, um beliebige Schaltfunktionen damit umzusetzen? Bilden Sie ein ODER nur aus NAND-Gattern.
- Wann setzen Sie ein PROM, wann ein PAL ein?
- Was ist ein Glitch, und welche Rolle spielt die Laufzeitverzögerung der Gatter dabei?
- Wie können Sie Glitches in Schaltnetzen vermeiden?

Ihre Begriffe für die Kurslandkarte

Die Kurslandkarte wächst mit dem Semester, und Sie bestimmen mit, was hineinkommt. Ihre Antworten sammeln wir im Hörsaal gemeinsam über das LiaScript-Klassenzimmer; die Begriffe dieser Woche sind seit Beginn der Vorlesung in der Karte freigeschaltet, und Ihre Vorschläge ergänzen sie.

Welche Begriffe aus der heutigen Vorlesung gehören für Sie in die Kurslandkarte?

Texteingabe ...

Wo begegnet Ihnen das, was heute dran war – in Ihrem Studiengang oder in Geräten Ihres Alltags?

Texteingabe ...

Hausaufgaben

Bis zum Beginn der Übungen Ende November ersetzt die Hausaufgabe die Übung. Planen Sie 60–75 Minuten ein. Teil A prüfen Sie direkt hier, Teil B besprechen wir zu Beginn von Vorlesung 06 am Mittwoch, 25.11.

Teil A – mit Selbstkontrolle

Aufgabe 1: Bestimmen Sie mit dem Verfahren von Quine-McCluskey alle Primimplikanten von $f(a,b,c) = \sum m(0,1,2,5,6,7)$, also $f=1$ in den Zeilen 0, 1, 2, 5, 6 und 7 der Wahrheitstafel.

Wie viele Primimplikanten hat f?

Wie viele davon sind essentiell?

Felder eintragen | Schleifen bilden | DNF (Einsen) | KNF (Nullen)

	$\bar{a}\bar{b}$	$\bar{a}b$	ab	$a\bar{b}$
$\bar{c}$	1 (0)	1 (2)	1 (6)	0 (4)
c	1 (1)	0 (3)	1 (7)	1 (5)

Schleife übernehmen | Lösung zeigen | Zurücksetzen

Felder anklicken, die eine Schleife bilden.

f = ?

Noch nicht überdeckt: 6 Einsen.

Aufgabe 2: Die Funktion $y = ab + cd$ soll nur aus NAND-Gattern mit zwei Eingängen aufgebaut werden. Die Eingänge liegen nur einfach vor, nicht negiert.

Wie viele NAND-Gatter brauchen Sie mindestens?

Aufgabe 3: Ein Schaltnetz hat 6 Eingänge und 3 Ausgänge und soll als PROM umgesetzt werden.

Wie viele Bit muss das PROM speichern?

Aufgabe 4: Die Funktion $f = a\bar{c} + bc$ wird zweistufig umgesetzt.

Felder eintragen | Schleifen bilden | DNF (Einsen) | KNF (Nullen)

	$\bar{a}\bar{b}$	$\bar{a}b$	ab	$a\bar{b}$
$\bar{c}$	0 (0)	0 (2)	1 (6)	1 (4)
c	0 (1)	1 (3)	1 (7)	0 (5)

Schleife übernehmen | Lösung zeigen | Zurücksetzen

Felder anklicken, die eine Schleife bilden.

f = ?

Noch nicht überdeckt: 4 Einsen.

Welcher zusätzliche Term verhindert einen Glitch?

- ○ ab
- ○ ac
- ○ $b\bar{c}$
- ○ keiner, f ist bereits glitchfrei

Teil B – Besprechung in Vorlesung 06

Aufgabe 5: Minimieren Sie $f(a,b,c,d) = \sum m(0,1,2,5,6,7,8,9,10,14)$ von Hand mit dem Verfahren von Quine-McCluskey. Stellen Sie die Primimplikantentafel auf, bestimmen Sie die essentiellen Primimplikanten und eine minimale Überdeckung. Prüfen Sie Ihr Ergebnis mit dem KV-Diagramm und mit SymPy.

Felder eintragen | Schleifen bilden | DNF (Einsen) | KNF (Nullen)

	$\bar{a}\,\bar{b}$	$\bar{a}\,b$	$a\,b$	$a\,\bar{b}$
$\bar{c}\,\bar{d}$	1 0	0 4	0 12	1 8
$\bar{c}\,d$	1 1	1 5	0 13	1 9
$c\,d$	0 3	1 7	0 15	0 11
$c\,\bar{d}$	1 2	1 6	1 14	1 10

Schleife übernehmen | Lösung zeigen | Zurücksetzen

Felder anklicken, die eine Schleife bilden.

f = ?

Noch nicht überdeckt: 10 Einsen.

Aufgabe 6: Segment d aus Vorlesung 03 soll in das PAL aus der Abbildung, mit 4 Produkttermen je Ausgang. Passt die minimierte Form mit Don't-Cares in einen Ausgang? Viele PAL-Bausteine haben invertierende Ausgänge, dann realisiert man $\bar{d}$ statt d. Minimieren Sie $\bar{d}$, ebenfalls mit Don't-Cares. Passt es jetzt?

Nächste Vorlesung

Am **Donnerstag, 19.11.**, geht es um Standardschaltnetze: Multiplexer, Dekoder, Kodierer und Komparatoren sind fertige Schaltnetze, aus denen sich größere Systeme bauen lassen.